==========

PASTIMES THREADS — SELF-REFLECTION REPORT

POE SECTION 2

==========

MODULE: Web Development (Intermediate) — WEDE6021/w

ASSESSMENT: Portfolio of Evidence (POE)

SUBMISSION DATE: June 2026

GROUP MEMBERS:

Clarity Masuku | ST10438928

Sibusiso Mabena | ST10462532

Unathi Mgandela | ST10447100

==========

1. INTRODUCTION

==========

This self-reflection report documents our experience building Pastimes Threads, a second-hand clothing marketplace web application developed as the Portfolio of Evidence (POE) for the WEDE6021 Web Development (Intermediate) module. The report covers what we built, the roles we each played, the research we conducted during development, our individual strengths and areas for improvement, and a

conclusion summarising our key takeaways from the project.

The POE required us to design and implement a fully functional e-commerce web application using PHP and MySQL that demonstrates an understanding of server-side scripting, database management, session-based authentication, shopping cart logic, and administration. We went beyond the minimum requirements by implementing a seller registration and approval system, a customer-to-admin messaging system, a customer complaint and support ticket system, and a per-tab session isolation feature to handle multi-account browser usage.

The application is built on PHP 8.2 (procedural), MySQL/MariaDB 10.4 via XAMPP, HTML5, CSS3, and vanilla JavaScript. There are no external frameworks — all functionality was written from scratch. This was a deliberate choice to ensure we understood every part of the stack and could explain how each feature works.

The project took us through the full development lifecycle: planning the database schema, building an authentication system, implementing the shopping cart with AJAX interactions, handling file uploads, writing an admin panel with full CRUD operations, and debugging real-world issues such as duplicate orders caused by browser refresh, SQL syntax errors in prepared statements, and session cookies being shared across browser tabs.

By the end of the project we had a working application that fulfils every rubric requirement: a startup page describing the eShop type and goals, a product grid with AddToCart and ShowCart buttons, cart persistence across page navigation, checkout with an order number and session ID reference, writes to the orderLine

table with stock decrements, an empty cart after checkout, and a purchase history report showing the grand total of all purchases. The application also executes cleanly from the index page and all core flows are demonstrated in our video submission.

To make our testing realistic and ensure every user journey was properly experienced, each team member took on a specific role within the live application — Administrator, Seller, and Customer — and used it throughout development and testing as a real user would. The sections below describe both the development contributions and the real application role each person owned.

==========

2. ROLE IN THE TEAM

==========

To ensure all three user journeys were tested by someone with personal ownership over that experience, we divided the application roles between team members. Clarity operated as the system Administrator, Sibusiso registered and operated as a Seller on the platform, and Unathi used the platform as a Customer/Buyer. This meant that every feature — from admin moderation to seller listing to customer checkout — was tested by someone with a stake in it working correctly from that user's point of view.

CLARITY MASUKU — ST10438928

APPLICATION ROLE: System Administrator (admin)

TEST ACCOUNT: clarity_admin | clarity@pastimes.co.za

As the Administrator on Pastimes Threads, Clarity had full access to the admin panel and was responsible for every management action the platform requires:

IN THE APPLICATION (what she did as admin during testing):

- Logged in using the admin account, which bypasses the shop entirely and lands directly on the admin dashboard.
- Reviewed and approved new customer registrations that arrived with a "pending" status. Without her approval, new users could not log in.
- Approved and rejected seller applications — she reviewed Sibusiso's seller registration and approved his account, allowing his products to appear in the shop for Unathi to purchase.
- Used the Products tab to add, edit, and delete clothing listings on behalf of the platform.
- Reviewed customer orders in admin_orders.php and updated order statuses from "pending" to "shipped" and "delivered" so that Unathi could see the status tracker on My Orders update in real time.
- Received and responded to messages sent by Unathi (the customer) through the admin messaging dashboard.
- Handled customer support tickets submitted by Unathi through the reports system, updated ticket statuses, and wrote admin replies.

AS A DEVELOPER (what she built):

- Designed the full database schema (9 tables) and wrote clothingstore.sql.
- Built the login, registration, and bcrypt password authentication system.
- Wrote the cart logic (cart_add.php, cart_remove.php, cart_update.php) and the full checkout and order creation flow in process_checkout.php.
- Implemented the PRG (Post/Redirect/Get) pattern that prevents duplicate orders when the browser is refreshed on the confirmation page.
- Built the admin panel (admin.php) with user and product CRUD, image upload validation, and the seller approval workflow.
- Debugged the $0.00 order total bug and the messages.php SQL crash.

SIBUSISO MABENA — ST10462532

APPLICATION ROLE: Seller

TEST ACCOUNT: sipho_threads | sipho@pastimes.co.za

As a Seller on Pastimes Threads, Sibusiso went through the full seller journey from first registration to listing products and watching those products reach customers:

IN THE APPLICATION (what he did as a seller during testing):

- Visited seller_register.php and submitted a seller application for his brand "Sipho Threads", listing the type of clothing he sells and his contact details.
- Waited for Clarity (admin) to approve the application — this gave us a

real test of the approval gate. Before approval, his products did not appear in the shop at all, even if they were inserted into the database.

- After approval, used the Seller Hub (sellers_hub.php) to add new clothing listings: uploaded product photos, set prices, wrote descriptions, and set stock quantities.
- Checked the Seller Hub dashboard to see which of his listings had been purchased and to track his total revenue.
- Sent a message through messages.php to the admin (Clarity) to request a change to one of his listings — testing the seller-to-admin communication channel.
- Verified that when Clarity (admin) marked his products as out of stock, they disappeared from the shop without needing to be deleted.

AS A DEVELOPER (what he built):

- Created the application-wide CSS design system: custom properties, colour palette, card layouts, responsive grid, and consistent typography.
- Built the shop.php product grid with the AJAX addToCart function, toast notification system, and live cart badge count.
- Designed and built checkout.php (ShowCart) with cart table, quantity controls, and real-time subtotal calculations.
- Built my_orders.php with the 3-step visual status tracker, summary bar, and expandable order cards including product thumbnails.
- Created pimg.php, the dynamic SVG product image generator that produces a professional-looking placeholder for any product without an uploaded photo.
- Designed report.php with category chips, order dropdown pre-fill, and the support ticket history sidebar.

UNATHI MGANDELA — ST10447100

APPLICATION ROLE: Customer / Buyer

TEST ACCOUNT: unathi_buys | unathi@pastimes.co.za

As a Customer on Pastimes Threads, Unathi went through every buying journey a real customer would experience — from first registration to receiving a delivered order and raising a complaint:

IN THE APPLICATION (what she did as a customer during testing):

- Registered a new account at register.php. Her account started as "pending" and she could not log in until Clarity (admin) approved it.
- Once approved, logged in and browsed the shop, where she could only see products from sellers that admin had approved (Sibusiso's listings appeared; an unapproved test seller's listings did not).
- Added multiple items to the cart — including adding the same item twice to confirm the quantity increased rather than creating a duplicate line.
- Navigated away from the shop to the home page and back — the cart was still intact because it is saved in the database, not in a session variable.
- Completed checkout, filling in a delivery address. The confirmation page showed her Order Number (ORD-00000001) and Session ID as reference numbers.
- Opened My Orders to see her order history, the 3-step status tracker, and the grand total of all her purchases at the top.
- Sent a message to admin through messages.php asking about her delivery, and

received a reply from Clarity (admin) that appeared in the conversation thread.
- Used report.php to submit a complaint about a delayed order — selected the order from the dropdown, chose "Order Issue" as the category, wrote a description, and tracked the admin's reply and status update in the history.
- Tested the cross-tab session issue: opened a second tab, logged in as a different user, and confirmed that her original tab detected the change and redirected to the login page with the session conflict message.

AS A DEVELOPER (what she built):
- Set up the XAMPP environment and confirmed the database name case-sensitivity requirement (ClothingStore with capital C and S).
- Built the seller registration and seller hub flows, including the LEFT JOIN approval filter on shop.php that gates unapproved seller products.
- Built the messaging system (messages.php and admin_messages.php), including fixing the SQL crash caused by using ? placeholders in a raw mysqli_query call.
- Built admin_reports.php with filter tabs, stat cards, and the reply form.
- Conducted end-to-end testing of all user flows and wrote the test case list.
- Wrote config/tab_guard.php, the sessionStorage-based per-tab session guard.

==========

3. RESEARCH, TECHNOLOGY AND PRESENTATION OF INFORMATION

==========

During development we encountered two significant technical challenges that

required research before we could implement a working solution.

SCENARIO 1: PHP Session Management and Cross-Tab Login Isolation

---

PROBLEM:

When Unathi (customer) and Clarity (admin) each had a browser tab open and Clarity logged into her admin account in a new tab, Unathi's customer tab immediately changed to show Clarity's account. This happened because PHP stores session data in a single file identified by a cookie (PHPSESSID) that is shared across all tabs in the same browser window. Any write to $_SESSION in one tab is instantly visible in all others, since they all read the same session file on disk.

This was a real problem during testing: if we demonstrated the admin panel and the customer shop side-by-side in two tabs, the accounts would corrupt each other.

RESEARCH:

We investigated three possible solutions:

(a) Separate session names per tab — not feasible because session_name() in PHP applies server-wide via php.ini; it cannot be set differently for individual tabs.

(b) localStorage in JavaScript — also shared across all tabs of the same

origin, so it has the identical problem as PHP sessions.

(c) sessionStorage in JavaScript — tab-specific by the browser spec. Each tab gets its own isolated sessionStorage that is never visible to other tabs and is automatically cleared when the tab is closed.

SOLUTION IMPLEMENTED:

We created config/tab_guard.php, a PHP file that emits a JavaScript block included on every authenticated page. On load, the script reads the current PHP session's user_id (echoed from PHP into a JS variable) and compares it to the value stored in that tab's sessionStorage under the key "pt_tab_uid". If the two values differ, the tab clears its stored key and redirects to login.php with the message: "Another account signed in on a different tab. Please sign in again."

Unathi retested: with the fix in place, her customer tab no longer changed when Clarity logged into admin in another tab — each tab maintained its own identity.

This scenario taught us the critical distinction between browser storage layers: PHP sessions and cookies are shared across all same-origin tabs; sessionStorage is strictly per-tab. Knowing which layer holds which state is fundamental to building secure multi-user applications.

SCENARIO 2: Cart Persistence — Session Array vs Database Table

---

PROBLEM:

Unathi (customer) reported that when she navigated away from the shop page and came back, she expected her cart to still have the items she had added. This raised a planning question: where should the cart be stored? We had two options and needed to research them before deciding.

RESEARCH:

We identified the following trade-offs:

SESSION ARRAY approach ($_SESSION['cart']):

- Simple to implement — no extra database table needed.
- Cart is lost if the session expires (XAMPP default: 24 minutes idle), the Apache server restarts, or the user clears their browser cookies.
- Two different devices (phone + laptop) cannot share the same cart.
- Difficult to query for admin sales reporting or analytics.
- Matching the rubric requirement — "cart remains available when continuing to shop" — is fragile because session expiry can silently zero the cart.

DATABASE TABLE approach (tblcart):

- Requires an extra table and a database round-trip on every cart operation.
- Cart survives browser close, session expiry, server restart, and device switching as long as the user logs back in with the same account.
- SQL aggregate queries make it trivial to count items, sum prices, and generate reports for admin.
- The ON DUPLICATE KEY UPDATE technique in MySQL allows a single INSERT to either add a new row (first add of a product) or increment the quantity

(re-adding the same product), without needing a SELECT-then-INSERT check.

SOLUTION IMPLEMENTED:

We chose the database table approach. tblcart has a unique composite key on (user_id, product_id), enforcing the rule that each user can have at most one row per product. The AJAX endpoint cart_add.php uses:

```
INSERT INTO tblcart (user_id, product_id, quantity)
VALUES (?, ?, 1)
ON DUPLICATE KEY UPDATE quantity = quantity + 1
```

Unathi tested: she added items to her cart, closed the browser tab entirely, reopened the application and logged back in, and found all her items still in the cart at the same quantities. The rubric criterion — "shopping cart remains available with selected items still intact when user continues shopping" — is met at the highest level because the cart is durable storage, not ephemeral memory.

==========

4. PERSONAL STRENGTHS AND WEAKNESSES

==========

CLARITY MASUKU — ST10438928 (Administrator role)

STRENGTHS:

1. Back-end problem solving — As the administrator of the live application, Clarity needed to understand every database transaction and server-side flow. This shaped how she wrote code: when the checkout total showed $0.00 on the confirmation page, she traced the full POST → redirect → GET lifecycle to find that the condition re-queried the already-emptied cart on the confirmation load, rather than applying a surface-level workaround.

2. Database design — Clarity designed a normalised schema with nine tables, appropriate constraints, and foreign keys before any PHP was written. The ON DUPLICATE KEY constraint on tblcart, the unique key that enforces one cart row per (user, product), and the approval_status column on tblseller were all planned from the start, which meant the logic was correct by schema design rather than by extra application code.

3. Security awareness — Every form and query Clarity wrote used prepared statements with bound parameters, bcrypt password hashing, and htmlspecialchars output escaping. As the admin-role tester she was also the most likely to notice if the role checks on shop.php or checkout.php could be bypassed, and she caught and fixed two missing admin redirects before testing began.

WEAKNESSES:

1. Front-end CSS — Clarity found CSS layout and responsive design time-consuming. Grid and flexbox alignment in edge cases (especially the admin panel's two-column product form) required multiple attempts. Going forward, spending dedicated time on CSS fundamentals — box model, flexbox axis alignment, and grid template areas — would reduce this friction.

2. Over-scoping features before core is done — During the first week Clarity built a multi-level seller approval workflow before the basic cart was finished. Learning to complete the minimum working version first and then layer additional complexity is an area she identified for improvement.

SIBUSISO MABENA — ST10462532 (Seller role)

STRENGTHS:

1. UI consistency and design system thinking — As the seller, Sibusiso needed every page to look and feel trustworthy, because sellers judge a platform by whether it looks professional enough to attract buyers. This motivated him to build a CSS design system at the start: custom properties, a fixed colour palette, and reusable card classes. The result was a consistent look across all 20 pages without any team member needing to reinvent the styles.

2. JavaScript interactivity from the seller's perspective — Sibusiso noticed

as a seller that the shop page gave no instant feedback when a buyer added an item — the page just seemed frozen for a moment. This is why he built the AJAX addToCart with the toast notification and live badge update: so buyers get immediate confirmation that their action worked. The experience of using the platform as a seller made him more sensitive to buyer-facing UX.

3. Practical creativity — When product image uploads were not available during early testing, Sibusiso built pimg.php: a PHP script that generates a professional-looking SVG image from the product name and brand, using colour gradients and the product's initial letter. This unblocked the entire team from waiting for real images before the shop could be tested visually.

WEAKNESSES:

1. PHP back-end confidence — When a PHP error appeared in a page Sibusiso was working on he would often ask Clarity to fix it rather than reading the error message and tracing it himself. Building comfort with PHP error output, checking $_SESSION state with var_dump, and reading mysqli error codes are skills he identified as areas to develop.

2. Git commit discipline — Several of Sibusiso's commits contained multiple unrelated changes under a single vague message such as "updates" or "fix". When a layout bug appeared in checkout.php, it was difficult to identify which commit introduced it. Writing focused commits with clear messages describing what changed and why is a practice he needs to build.

UNATHI MGANDELA — ST10447100 (Customer role)

STRENGTHS:

1. Thoroughness as a tester from the customer's viewpoint — Because Unathi lived the full customer journey — registration, waiting for approval, shopping, checkout, order tracking, messaging admin, filing a complaint — she caught bugs that would only surface in that exact sequence. She found the cross-tab session issue when she had a customer tab open and Clarity logged into admin in a second tab. She also found that navigating directly to checkout.php with an empty cart showed a broken page, which led to an empty-cart guard being added.

2. Environment and documentation — Unathi managed the XAMPP setup for all team members. When the application threw a fatal error on first run due to the database being created as "clothingstore" instead of "ClothingStore", she identified the case-sensitivity issue, documented the exact fix, and updated the setup instructions so it would not happen again.

3. Understanding cross-system integration — The seller approval gate on shop.php (the LEFT JOIN with approval_status filter) was Unathi's solution to a problem she discovered as a customer: products from an unapproved seller were appearing in the shop. Understanding how tblclothes and tblseller related to each other, and how to filter across that join, showed

that her database knowledge extended beyond individual table operations to multi-table reasoning.

WEAKNESSES:

1. CSS layout confidence — Like Clarity, Unathi found front-end styling less comfortable than back-end work. When building admin_reports.php she rebuilt the filter tab layout three times before it matched the rest of the application. Reading the existing shop.php and my_orders.php CSS more carefully before starting new pages — and reusing the established patterns — would save significant time.

2. Tendency to perfect one feature before moving on — Unathi would sometimes spend extra time perfecting one area (the messaging UI, the report ticket form) before testing the next feature. This occasionally meant features near the end of the project had less testing time. A "build it working, then improve it" approach spread across all features would give better overall coverage.

==========

5. CONCLUSION

==========

Building Pastimes Threads was one of the most practically valuable projects we

have completed during this module. It moved us from writing individual PHP scripts to building a multi-role, multi-user web application with a real database, real session management concerns, and real security requirements.

The decision to assign each team member a real application role — Administrator, Seller, and Customer — transformed our testing from a checklist exercise into an experience. Clarity managing the platform as admin, Sibusiso going through the seller onboarding and listing process, and Unathi completing the full customer buying journey meant that bugs were discovered in the same way a real user would encounter them, not by reading the code but by using the product.

Every bug we fixed was a lesson: the $0.00 checkout total taught us the PRG pattern; the cross-tab session issue taught us the difference between shared and tab-local browser storage; the messages.php SQL crash taught us that raw mysqli_query() cannot use prepared-statement placeholders; the missing seller approval gate taught us that role-based data filtering must happen in the query, not just in the UI. Each fix made us more capable developers.

The additional features we built — the seller hub with approval gating, the admin messaging and report-management systems, the per-tab session guard, and the dynamic SVG product images — went beyond the brief and demonstrated that we understood not just the mechanics of PHP and MySQL but the design thinking behind a real e-commerce platform: who are the users, what do they need, and how do we protect each user from the others?

We are confident that Pastimes Threads meets and exceeds the WEDE6021 POE

requirements, and that the challenges we faced and the way we solved them are clearly reflected in this self-reflection.

==========

MODULE: Web Development (Intermediate) — WEDE6021/w

END OF SELF-REFLECTION

==========